LAKSHMI NARAIN COLLEGE OF TECHNOLOGY

Department of Computer Science & Engineering

ASSIGNMENT REPORT

Submitted By

VARUN JUNEJA

College	**LNCT**
Enrollment Number	**0103CS231449**
Superset ID	**7809995**
Email	**varunjuneja1905@gmail.com**

Academic Session 2025 – 2026

EXERCISE 1: CONTROL STRUCTURES

Scenario 1: Discount for Senior Citizens

```
DECLARE
    CURSOR c_customers IS
        SELECT c.CustomerID, l.LoanID, l.InterestRate,
               FLOOR(MONTHS_BETWEEN(SYSDATE, c.DOB)/12) Age
        FROM Customers c
        JOIN Loans l ON c.CustomerID = l.CustomerID;

BEGIN
    FOR rec IN c_customers LOOP
        IF rec.Age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE LoanID = rec.LoanID;
        END IF;
    END LOOP;

    COMMIT;
END;
/
```

Scenario 2: Promote Customers to VIP

```
ALTER TABLE Customers ADD IsVIP VARCHAR2(5);

BEGIN
    FOR rec IN (SELECT CustomerID, Balance FROM Customers) LOOP
        IF rec.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = rec.CustomerID;
        END IF;
    END LOOP;

    COMMIT;
END;
/
```

Scenario 3: Loan Due Reminder

```
BEGIN
    FOR rec IN (
        SELECT c.Name, l.EndDate
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID = l.CustomerID
        WHERE l.EndDate BETWEEN SYSDATE
              AND SYSDATE + 30
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Reminder: Loan due for '
            || rec.Name ||
            ' on ' || rec.EndDate);
    END LOOP;
END;
/
```

EXERCISE 2: ERROR HANDLING

Scenario 1: SafeTransferFunds

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_from_acc NUMBER,
    p_to_acc NUMBER,
    p_amount NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_acc;

    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Insufficient Balance');
    END IF;

    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_acc;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_acc;

    COMMIT;
```

```
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```

Scenario 2: UpdateSalary

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_emp_id NUMBER,
    p_percent NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary +
                 (Salary * p_percent / 100)
    WHERE EmployeeID = p_emp_id;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE(
            'Employee not found');
END;
/
```

Scenario 3: AddNewCustomer

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_id NUMBER,
    p_name VARCHAR2,
    p_dob DATE,
    p_balance NUMBER
)
IS
BEGIN
    INSERT INTO Customers
    VALUES(p_id,p_name,p_dob,
           p_balance,SYSDATE);
```

```
    COMMIT;

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE(
            'Customer ID already exists');
END;
/
```

EXERCISE 3: STORED PROCEDURES

Scenario 1: ProcessMonthlyInterest

```
CREATE OR REPLACE PROCEDURE
ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType='Savings';

    COMMIT;
END;
/
```

Scenario 2: UpdateEmployeeBonus

```
CREATE OR REPLACE PROCEDURE
UpdateEmployeeBonus(
    p_department VARCHAR2,
    p_bonus NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary +
                 (Salary * p_bonus /100)
    WHERE Department = p_department;

    COMMIT;
END;
/
```

Scenario 3: TransferFunds

```
CREATE OR REPLACE PROCEDURE
TransferFunds(
    p_from NUMBER,
    p_to NUMBER,
    p_amount NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from;

    IF v_balance >= p_amount THEN

        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_from;

        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_to;

        COMMIT;
    ELSE
        DBMS_OUTPUT.PUT_LINE(
            'Insufficient Balance');
    END IF;
END;
/
```

EXERCISE 4: FUNCTIONS

Scenario 1: CalculateAge

```
CREATE OR REPLACE FUNCTION
CalculateAge(
    p_dob DATE
)
RETURN NUMBER
IS
BEGIN
    RETURN FLOOR(
        MONTHS_BETWEEN(
            SYSDATE,
            p_dob)/12);
```

```
END;
/
```

Scenario 2: CalculateMonthlyInstallment

```
CREATE OR REPLACE FUNCTION
CalculateMonthlyInstallment(
    p_loan NUMBER,
    p_rate NUMBER,
    p_years NUMBER
)
RETURN NUMBER
IS
    v_months NUMBER;
BEGIN
    v_months := p_years * 12;

    RETURN (p_loan +
            (p_loan*p_rate/100))
            / v_months;
END;
/
```

Scenario 3: HasSufficientBalance

```
CREATE OR REPLACE FUNCTION
HasSufficientBalance(
    p_acc_id NUMBER,
    p_amount NUMBER
)
RETURN BOOLEAN
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Accounts
    WHERE AccountID = p_acc_id;

    RETURN v_balance >= p_amount;
END;
/
```

EXERCISE 5: TRIGGERS

Scenario 1: UpdateCustomerLastModified

```
CREATE OR REPLACE TRIGGER
UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/
```

Scenario 2: LogTransaction

```
CREATE TABLE AuditLog(
    LogID NUMBER GENERATED ALWAYS AS IDENTITY,
    TransactionID NUMBER,
    LogDate DATE
);

CREATE OR REPLACE TRIGGER
LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog(
        TransactionID, LogDate)
    VALUES(
        :NEW.TransactionID,
        SYSDATE);
END;
/
```

Scenario 3: CheckTransactionRules

```
CREATE OR REPLACE TRIGGER
CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Accounts
    WHERE AccountID = :NEW.AccountID;
```

```
    IF :NEW.TransactionType='Withdrawal'
       AND :NEW.Amount > v_balance THEN

        RAISE_APPLICATION_ERROR(
            -20002,
            'Withdrawal exceeds balance');

    ELSIF :NEW.TransactionType='Deposit'
          AND :NEW.Amount <= 0 THEN

        RAISE_APPLICATION_ERROR(
            -20003,
            'Deposit must be positive');
    END IF;
END;
/
```

EXERCISE 6: CURSORS

Scenario 1: GenerateMonthlyStatements

```
DECLARE
    CURSOR c_stmt IS
        SELECT * FROM Transactions
        WHERE EXTRACT(MONTH FROM TransactionDate)
              = EXTRACT(MONTH FROM SYSDATE);

BEGIN
    FOR rec IN c_stmt LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Account: ' || rec.AccountID ||
            ' Amount: ' || rec.Amount);
    END LOOP;
END;
/
```

Scenario 2: ApplyAnnualFee

```
DECLARE
    CURSOR c_acc IS
        SELECT AccountID FROM Accounts;

BEGIN
    FOR rec IN c_acc LOOP
        UPDATE Accounts
        SET Balance = Balance - 100
```

```
        WHERE AccountID = rec.AccountID;
    END LOOP;

    COMMIT;
END;
/
```

Scenario 3: UpdateLoanInterestRates

```
DECLARE
    CURSOR c_loan IS
        SELECT LoanID, InterestRate
        FROM Loans;

BEGIN
    FOR rec IN c_loan LOOP
        UPDATE Loans
        SET InterestRate =
            rec.InterestRate + 0.5
        WHERE LoanID = rec.LoanID;
    END LOOP;

    COMMIT;
END;
/
```

EXERCISE 7: PACKAGES

Scenario 1: CustomerManagement Package

```
CREATE OR REPLACE PACKAGE CustomerManagement AS

    PROCEDURE AddCustomer(
        p_id NUMBER,
        p_name VARCHAR2);

    PROCEDURE UpdateCustomer(
        p_id NUMBER,
        p_balance NUMBER);

    FUNCTION GetBalance(
        p_id NUMBER)
    RETURN NUMBER;

END;
/
```

```
CREATE OR REPLACE PACKAGE BODY
CustomerManagement AS

PROCEDURE AddCustomer(
    p_id NUMBER,
    p_name VARCHAR2)
IS
BEGIN
    INSERT INTO Customers
    (CustomerID,Name,LastModified)
    VALUES(p_id,p_name,SYSDATE);
END;

PROCEDURE UpdateCustomer(
    p_id NUMBER,
    p_balance NUMBER)
IS
BEGIN
    UPDATE Customers
    SET Balance = p_balance
    WHERE CustomerID = p_id;
END;

FUNCTION GetBalance(
    p_id NUMBER)
RETURN NUMBER
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Customers
    WHERE CustomerID = p_id;

    RETURN v_balance;
END;

END;
/
```

Scenario 2: EmployeeManagement Package

```
CREATE OR REPLACE PACKAGE EmployeeManagement AS

    PROCEDURE HireEmployee(
        p_id NUMBER,
        p_name VARCHAR2);

    PROCEDURE UpdateEmployee(
        p_id NUMBER,
        p_salary NUMBER);
```

```
    FUNCTION AnnualSalary(
        p_id NUMBER)
    RETURN NUMBER;

END;
/
```

Scenario 3: AccountOperations Package

```
CREATE OR REPLACE PACKAGE AccountOperations AS

    PROCEDURE OpenAccount(
        p_acc NUMBER,
        p_cust NUMBER);

    PROCEDURE CloseAccount(
        p_acc NUMBER);

    FUNCTION TotalBalance(
        p_cust NUMBER)
    RETURN NUMBER;

END;
/
```